Universidade Federal de Viçosa – Campus Florestal
Projeto Integrador 2025 - Arquitetura de Software
Equipe 2

Adicionando Áudios Narrativos na Arquitetura

Este documento pretende explicar a estrutura para adicionar arquivos de áudio narrativos como ativos estáticos do backend do jogo “Pingo: Uma Aventura da Rede”, idealizado e desenvolvido pela Equipe 2 durante o Projeto Integrador 2025.

Data: 27/10/2025
Autor(a): Ana Carolina Fernandes, Arquiteta de Software

1. Visão Geral **2**
2. Comportamento das Camadas **2**
2.1. Model (Camada de Domínio) 2
2.2. Data Access Object (Camada de Persistência) 3
2.3. Service (Camada de Negócio) 3
2.4. Controller (Camada de Apresentação) 4
3. Conclusão **4**

1. Visão Geral

Esta abordagem está considerando que os áudios são parte do código-fonte do projeto, o que significa que os arquivos são parte do código-fonte, gerenciados pela equipe de desenvolvimento.

Os arquivos de áudio narrativos devem ser colocados diretamente na pasta *resources* do projeto. O local padrão para os áudios narrativos será:

```
None
pingo/
├── src/
│   └── main/
│   │    └── resources/
│   │    │    └── static/
│   │    │    │    └── audios/
│   │    │    │    │    └── narracao/
│   │    │    │    │    │    └── fase1_narracao.mp3
```

É importante ressaltar que os arquivos ficarão publicamente disponíveis para o frontend na URL: *http://localhost:8080/audios/narracao/fase1_narracao.mp3*

2. Comportamento das Camadas

O fluxo seguirá a arquitetura definida anteriormente para o projeto.

2.1. Model (Camada de Domínio)

Onde ficará a entidade que armazenará a URL do áudio e os demais dados necessários.

Exemplo de implementação:

```java
@Entity
@Table(name = "stage_audios")
public class StageAudio {
	@Id private Long id;

	@Column(nullable = false)
	private String filepath;

	// Demais atributos

}
```

2.2. Data Access Object (Camada de Persistência)

Esta camada será responsável por buscar os metadados do áudio no banco de dados.

2.3. Service (Camada de Negócio)

Essa camada terá a lógica para buscar os áudios associados a uma determinada fase.

Exemplo de implementação:

```java
@Service
public class StageService {

    private final StageAudioDAO stageAudioDAO;

    public StageService(StageAudioDAO stageAudioDAO) {
        this.stageAudioDAO = stageAudioDAO;
    }

    public List<StageAudio> getAudiosForStage(Long stageId) {
        if (stageId == null || stageId <= 0) {
             throw new InvalidParameterException("ID de fase inválido.");
```

```
        }
        return stageAudioDAO.findAllByStageId(stageId);
    }
}
```

2.4. Controller (Camada de Apresentação)

Nessa camada terá um endpoint *GET* para retornar a lista de áudios daquela fase para o frontend.

Exemplo de implementação:

```
Java
@RestController
@RequestMapping("/api/stages")
public class StageController {
    private final StageService stageService;

       public StageController(StageService stageService) {
              this.stageService = stageService;
       }

    @GetMapping("/{stageId}/audios")
    public ResponseEntity<List<StageAudio>> getAudiosForStage(@PathVariable
Long stageId) {
        List<StageAudio> audios = stageService.getAudiosForStage(stageId);
        return ResponseEntity.ok(audios);
    }
}
```

3. Conclusão

A partir deste arquivo o fluxo para a adição de áudios no projeto deve inicialmente fazendo a adição dos arquivos de áudio na pasta *src/main/resources/static/audios/*, e inserindo os registros de metadados correspondentes (com o filepath público, como */audios/narracao/fase1.mp3*) no banco de dados, podendo ser via sql. E por fim, é necessário a implementação das

camadas de Controller e Service para consultar e retornar esses caminhos ao frontend.